Concurrent Programming

Basic Concepts

1. What is the definition of concurrent programming?
   a) Programming that runs on a single processor only
   b) A way to structure a program to handle multiple tasks at once (or appear to)
   c) Programming that works without using memory
   d) Programming that only works on old operating systems
   Correct Answer: b
2. What is the main difference between concurrency and parallelism?
   a) Concurrency needs multiple processors while parallelism doesn't
   b) Parallelism needs multiple processors while concurrency doesn't necessarily
   c) There is no difference between them
   d) Concurrency is faster than parallelism
   Correct Answer: b
3. What is the main benefit of using concurrent programming?
   a) Reducing memory usage
   b) Improving performance and response time
   c) Making code more complicated
   d) Reducing program speed
   Correct Answer: b

Comparisons

4. What's the difference between sequential and concurrent execution?
   a) Sequential is faster than concurrent
   b) Concurrent executes tasks in sequence while sequential executes them together
   c) Sequential executes tasks one after another while concurrent can execute them together

d) There is no difference

Correct Answer: c

5. In the cooking analogy, what does a single chef preparing multiple dishes represent?

   a) Parallelism

   b) Concurrency

   c) Sequential execution

   d) Distributed processing

   Correct Answer: b

Threads

Thread Basics

6. What is a thread in programming?

   a) A lightweight unit of execution within a process

   b) A type of memory

   c) A new programming language

   d) A tool for memory cleanup

   Correct Answer: a

7. What do threads within the same process share?

   a) Memory and resources

   b) Only processor registers

   c) Only the stack

   d) Nothing

   Correct Answer: a

8. What is the main difference between a process and a thread?

   a) Threads don't need resources

   b) Processes share memory while threads don't

   c) Threads share memory while processes don't

   d) There is no difference

   Correct Answer: c

Thread Management

9. What is the purpose of join() in Python threads?
   a) To terminate the thread immediately
   b) To make the main thread wait for the child thread to complete
   c) To increase thread speed
   d) To reduce memory usage
   Correct Answer: b
10. What happens if you don't use join() with threads?
   a) The program will stop working
   b) The main thread may finish before child threads
   c) Threads will run twice as fast
   d) Nothing will be different
   Correct Answer: b

Daemon Threads

11. What are daemon threads?
   a) Threads that terminate when the main thread ends
   b) Threads that cannot be stopped
   c) Threads that only work during daytime
   d) Threads that are faster than normal threads
   Correct Answer: a
12. When should you use daemon threads?
   a) For critical tasks that must complete
   b) For background tasks that can be abandoned
   c) Only in web applications
   d) When you want to increase memory usage
   Correct Answer: b
13. How do you convert a normal thread to a daemon thread?
   a) t.daemon = True before start()
   b) t.daemon = False after start()
   c) t.stop() when finished
   d) t.join() at the beginning
   Correct Answer: a

Synchronization

Race Conditions

14. What is a race condition?
    a) When a thread wins a speed competition
    b) When multiple threads compete for a shared resource without coordination
    c) When threads run very slowly
    d) When threads refuse to work together
    Correct Answer: b
15. What causes race conditions?
    a) Using shared memory without synchronization
    b) Using daemon threads
    c) Not having enough threads
    d) Using join() too much
    Correct Answer: a

Locks

16. What is the purpose of using locks?
    a) To make the program slower
    b) To prevent race conditions by controlling access to shared resources
    c) To increase the number of threads
    d) To reduce memory usage
    Correct Answer: b
17. How do you use a lock in Python?
    a) with lock:
    b) lock.start()
    c) lock.join()
    d) lock.delete()
    Correct Answer: a
18. What is a critical section?
    a) The part of code that accesses shared resources
    b) The hardest part of the program

c) The part that only runs on one thread

d) The part that cannot be modified

Correct Answer: a

Practical Examples

19. In the "juice bottle problem" example, what prevents the race condition?

    a) Using join()

    b) Using a Lock

    c) Increasing the number of threads

    d) Reducing sleep time

    Correct Answer: b

20. If you have 3 files to download, what's the approximate execution time using threads?

    a) The sum of all file download times

    b) The time of the longest file

    c) The time of the shortest file

    d) The average of all file times

    Correct Answer: b

Advanced Concepts

21. What is the memory model in concurrent programming?

    a) How threads interact with shared memory

    b) The amount of memory required by the program

    c) A language for specifying memory

    d) A tool for analyzing memory usage

    Correct Answer: a

22. What are performance hazards in threads?

    a) Increasing program speed

    b) Performance problems due to excessive concurrency

    c) Reducing memory usage

    d) There are no performance hazards

    Correct Answer: b

23. What is the best way to design a thread-safe program?
    a) Design it to be thread-safe from the beginning
    b) Modify it later to become thread-safe
    c) Use only daemon threads
    d) Never use threads
    Correct Answer: a

Practical Applications

24. Where is concurrent programming used in real applications?
    a) Web servers to handle multiple requests
    b) User interfaces to stay responsive
    c) Background data loading
    d) All of the above
    Correct Answer: d
25. What is an example of a suitable task for daemon threads?
    a) Auto-saving documents in a word processor
    b) Processing payments in an e-commerce site
    c) Login functionality
    d) Updating the main user interface
    Correct Answer: a

History and Evolution

26. What was the first type of multitasking in operating systems?
    a) Cooperative multitasking
    b) Preemptive multitasking
    c) Multiprogramming
    d) Parallel processing
    Correct Answer: a
27. How did multitasking evolve in computers?
    a) From cooperative to preemptive
    b) From preemptive to cooperative
    c) It never changed

d) It disappeared completely

Correct Answer: a

Software Design

28. What is an advantage of using threads in software design?

    a) Simplifying the programming model

    b) Making code more complicated

    c) Reducing performance

    d) Only increasing memory usage

    Correct Answer: a

29. What is the main challenge in multithreaded programming?

    a) Understandability and predictability

    b) Execution speed

    c) Memory size

    d) The programming language used

    Correct Answer: a

Thread Safety

30. What does "thread-safe" code mean?

    a) It works correctly with multiple thread usage

    b) It's very fast

    c) It uses little memory

    d) It only works on one thread

    Correct Answer: a

31. How can you make variables thread-safe?

    a) By not sharing them between threads

    b) By making them immutable

    c) By using synchronization when accessing them

    d) All of the above

    Correct Answer: d

Code Examples

32. In the greeting threads example, how long does it take to execute greet() twice with two threads?
    a) 4 seconds
    b) 2 seconds
    c) 1 second
    d) 8 seconds
    Correct Answer: b
33. In the file download example, if each download takes 2 seconds, how long does it take to download 3 files with threads?
    a) 6 seconds
    b) 2 seconds
    c) 4 seconds
    d) 1 second
    Correct Answer: b

Resource Management

34. What resource do threads in the same process share?
    a) Memory
    b) Open files
    c) Network sockets
    d) All of the above
    Correct Answer: d
35. Why is resource sharing a problem in threads?
    a) It can lead to race conditions
    b) It increases program speed
    c) It reduces memory usage
    d) It makes code clearer
    Correct Answer: a

Performance Optimization

36. When is using threads beneficial for performance?
    a) When there are tasks that can run concurrently

b) When there are multiple processors

c) When there are I/O-bound tasks

d) All of the above

Correct Answer: d

37. What is the danger of overusing threads?

a) Increased context switching overhead

b) Higher memory consumption

c) More complex resource management

d) All of the above

Correct Answer: d

Common Misconceptions

38. Which of these statements about threads is false?

a) Threads always provide performance improvement

b) Threads share the process's memory

c) Threads need synchronization when sharing resources

d) Threads can cause race conditions

Correct Answer: a

39. What is a common misconception about concurrency and parallelism?

a) That they are the same thing

b) That parallelism doesn't need multiple processors

c) That concurrency is slower than sequential execution

d) That concurrency is easier to understand than sequential

Correct Answer: a

Final Question

40. What quote summarizes the challenge of multithreaded programming?

a) "Threads discard the most essential properties of sequential computation"

b) "Threads make everything easier"

c) "Threads provide no benefits"

d) "Threads are the only future of programming"

Correct Answer: a